AI-Powered Social Support Workflow Automation

Solution Summary
October 18, 2025

1. Problem Statement Recap

The current application process for the government social security department, responsible for financial and economic enablement support, faces significant inefficiencies. Processing times range from **5 to 20 working days**, hindering timely support for needy applicants. Key pain points include:

- **Manual Data Gathering:** Reliance on scanned documents, physical collection, and handwritten forms leads to data entry errors and delays.
- **Semi-Automated Validation:** Basic field checks coupled with extensive manual review for inconsistencies require significant effort.
- **Inconsistent Information:** Discrepancies often exist between various submitted documents (e.g., addresses, income, family details).
- **Time-Consuming Reviews:** Multiple departments and stakeholders involved in reviews create bottlenecks.
- **Subjective Decision-Making:** Human bias can lead to inconsistent and potentially unfair allocation of support.

2. Proposed Solution: Overview

This project implements an AI-driven workflow designed to automate approximately 99% of social support application decisions, aiming for completion within minutes of interaction. The solution leverages:

- **Agentic Orchestration:** A LangGraph state machine manages the workflow reliably.
- **Multimodal AI:** Local LLMs hosted via Ollama (Qwen2-VL, Llama 3.1) handle data extraction from various document types (images, PDFs, Excel).
- **Automated Validation:** Pydantic schemas and custom rules ensure data integrity.
- **Specialized Data Persistence:** A multi-database strategy (PostgreSQL, MongoDB, Neo4j, Qdrant) stores data optimally.
- **Explainable ML:** A Scikit-learn model provides objective eligibility scoring.
- **RAG for Enablement:** Vector search via Qdrant facilitates personalized job and training recommendations.
- **Observability:** Langfuse Cloud provides end-to-end tracing for transparency and debugging.
- **Interactive UI:** A Streamlit application serves as the user interface.

---

3. Solution Design

3.1 High-Level Architecture

```
graph TD
    subgraph User Interface
        U(Applicant) -- Interacts/Uploads --> FE(Streamlit App @ :8501)
    end

    subgraph API & Orchestration Layer
        FE -- HTTP Requests --> API(FastAPI Backend @ :8000)
        API -- Triggers Workflow --> ORC(LangGraph Orchestrator)
        ORC -- Sends Traces --> OBS(Langfuse Cloud)
    end

    subgraph Agentic AI Core [LangGraph State Machine]
        ORC -- Manages State --> A2(Data Extraction Agent)
        A2 -- Data/Error --> ORC
        ORC --> A3(Data Validation Agent)
        A3 -- Validated/Error --> ORC
        ORC --> A4(Data Persistence Agent)
        A4 -- Success/Error --> ORC
        ORC --> A5(Eligibility Agent)
```

```
    A5 -- Prediction/Error --> ORC
    ORC --> A6(Recommendation Agent)
    A6 -- Recommendation/Error --> ORC
    ORC --> A7(Save Result Agent)
    A7 -- Final Status --> ORC
    ORC --> END((End))
end

subgraph Local Model Hosting
    A2 -- Parser LLM Call --> OLLAMA(Ollama @ :11434\n- qwen2-vl:7b\n- lla
    A4 -- Embedding Call --> ST(SentenceTransformer\n'all-MiniLM-L6-v2')
end

subgraph Data & Persistence Layer
    A4 -- (Raw JSON) --> DB_MONGO(MongoDB @ :27017)
    A4 & A7 -- (Golden Record) --> DB_PG(PostgreSQL @ :5432)
    A4 -- (Relationships) --> DB_NEO4J(Neo4j @ :7687)
    A4 & A6 -- (Embeddings/RAG) --> DB_QDRANT(Qdrant @ :6333)
end

subgraph Classical ML Service
    A5 -- Feature Vector --> ML_API(FastAPI @ :8001)
    ML_API -- Loads --> MODEL(SK-Learn Model\n*.joblib)
    ML_API -- Prediction --> A5
end

subgraph Caching
    API -- Session/Cache --> DB_REDIS(Redis @ :6379)
end
```

Brief Component Explanation: The system ingests data via a Streamlit UI -> FastAPI backend. LangGraph orchestrates agents (Extraction, Validation, Persistence, Eligibility, Recommendation, Save Result) that use local Ollama models, Sentence Transformers, specialized databases, and a separate ML service. Langfuse Cloud captures traces.

3.2 Detailed Workflow & Data Flow

1. **Ingestion:** Applicant uses Streamlit UI to submit details and upload documents (ID, statement, resume, assets). FastAPI receives these and saves files locally.

2. **Trigger:** The final file upload triggers a background task in FastAPI, initiating the LangGraph workflow with the application details and file paths.

3. **Extraction (`extract_data`):**

    - Iterates through uploaded files.
    - Calls `document_parser.py` :
        - Uses `pdfplumber` for text from digital PDFs, then Ollama (`llama3.1:8b`) for JSON structuring.
        - Uses Ollama VLM (`qwen2-vl:7b`) for scanned PDFs/images.
        - Uses `pandas` for Excel files.
    - Saves raw output (success or error) to **MongoDB**.
    - Passes merged extracted data (`extracted_data`) to the next state.

4. **Validation (`validate_data`):**

    - Validates `extracted_data` against the `ValidatedApplicationData` Pydantic schema (types, required fields).
    - Performs basic custom checks (e.g., income presence if employed).
    - Passes validated/coerced data (`validated_data`) or sets an error message.

5. **Persistence (`persist_data`):**

    - If validation succeeded:
        - **PostgreSQL:** Updates `applications` table (JSONB `validated_data`) and `applicants` table.
        - **Neo4j:** Creates/updates `Person` , `Address` nodes and `:LIVES_AT` , `:HAS_FAMILY_MEMBER` relationships.

- **Qdrant:** If resume text exists, generates embedding (SentenceTransformer) and upserts vector into `applicant_resumes` collection.

6. **Eligibility Check (`check_eligibility`):**
   - Prepares features (e.g., income, family size) from `validated_data` .
   - Calls the dedicated ML Service API (`POST http://ml_service:8001/predict`) via `httpx` .
   - Receives prediction (`decision` , `probability`) from the Scikit-learn model.
   - Updates state with `eligibility_decision` and `eligibility_score` .
7. **Recommendation (`generate_recommendation`):**
   - Crafts a financial support message based on `eligibility_decision` .
   - Performs a placeholder RAG query against **Qdrant** (retrieving the applicant's own vector in V1) if eligible or declined.
   - Generates a placeholder economic enablement message.
   - Combines messages into `final_recommendation` .
8. **Save Result (`save_final_result`):**
   - Updates the `applications` table in **PostgreSQL** with the `final_recommendation` text and the final status (`COMPLETED` or `ERROR`).
9. **Completion:** The LangGraph workflow ends.
10. **Result Retrieval (UI):** Streamlit polls a `GET /result` endpoint on FastAPI, which queries PostgreSQL for the final status and recommendation to display to the user.
11. **Observability:** Each agent function decorated with `@observe` sends trace data (inputs, outputs, errors, metadata) to **Langfuse Cloud**.

3.3 Technology Justification

- **Core (Python, FastAPI, Streamlit):** Python meets the language requirement. FastAPI provides a high-performance, asynchronous API suitable for handling concurrent users and background tasks. Streamlit enables rapid development of the required interactive UI.
- **Containerization (Docker/Compose):** Ensures a consistent development and deployment environment across all services (databases, models, APIs), simplifying setup and dependency management. **Scalability:** Individual services (API, ML) can be scaled independently.
- **Orchestration (LangGraph):** Chosen for its explicit state management and ability to model complex, potentially cyclical workflows (useful for future validation loops). Its integration with Langfuse provides clear observability. **Maintainability:** Defining workflow as a graph improves clarity.
- **Local Models (Ollama):** Directly fulfills the requirement for locally hosted LLMs. Simplifies deployment of open-source models (Llama 3.1, Qwen2-VL) via a consistent API. **Security/Privacy:** Keeps sensitive data processing local.
- **Parsing Strategy (Ollama Client, pdfplumber, pandas):** Leverages multimodal LLMs directly for parsing complex documents (images/scans), providing flexibility. Uses `pdfplumber` as an efficient first-pass for digital PDFs. `pandas` handles structured Excel data reliably. This V1 approach prioritized rapid development and avoided problematic installations of heavy OCR libraries. **Performance:** Text-first approach for PDFs is fast; VLM calls are slower but handle difficult cases.
- **ML Model (Scikit-learn - HistGradientBoosting):** Provides a fast, robust, and *explainable* baseline for tabular eligibility prediction, directly addressing the “subjective decision-making” pain point. Serving via a separate microservice ensures modularity. **Maintainability:** Clear separation from the main agent logic.
- **Databases:**
    - *PostgreSQL:* Best fit for structured, relational data requiring ACID compliance (applicant profiles, application status, final results).

Scalability/Maintainability: Mature, well-supported, good ORM integration (SQLAlchemy).

- *MongoDB:* Ideal for storing semi-structured, evolving raw outputs from the extraction agent without requiring a predefined schema. **Scalability:** Horizontally scalable.

- *Neo4j:* Uniquely suited for modeling and querying relationships, crucial for detecting inconsistencies (e.g., multiple families, address conflicts) that are difficult to spot in other database types. **Suitability:** Directly addresses the “Inconsistent Information” pain point.

- *Qdrant:* High-performance vector database optimized for semantic similarity search needed for the RAG-based economic enablement feature. **Performance:** Essential for fast RAG lookups.

- *Redis:* Standard choice for caching API responses or short-term state, improving **Performance**.

- **Embeddings (Sentence Transformers):** Provides good quality embeddings locally on CPU, suitable for the RAG prototype without requiring external APIs or complex GPU setup.

- **Observability (Langfuse Cloud):** Meets the requirement. Cloud version used due to self-hosting difficulties under time constraints, providing robust tracing with minimal setup overhead. **Maintainability:** Offloads observability infrastructure management.

3.4 Agent Component Breakdown

- **`extract_data`:** Responsible for interfacing with `document_parser` to process all uploaded files (multimodal), saving raw results to MongoDB, and passing structured data to the state.

- **`validate_data`:** Enforces data quality using Pydantic schemas and applies business logic rules to check consistency across extracted fields.

- **`persist_data` :** Acts as a data router, saving validated information to the appropriate specialized database (Postgres, Neo4j) and generating/saving embeddings to Qdrant.

- **`check_eligibility` :** Prepares the feature set from validated data and calls the external ML microservice to get a score and decision.

- **`generate_recommendation` :** Synthesizes the final output, combining the financial decision with economic enablement suggestions derived (in V1, via placeholder) from RAG using Qdrant.

- **`save_final_result` :** Performs the final update to the primary PostgreSQL record, marking the application as completed or errored and storing the user-facing message.

---

4. Key Features & Pain Point Resolution

- **Automation (`Manual Data Gathering` , `Time-Consuming Reviews`):** AI agents replace manual data entry and streamline the review process.

- **Accuracy (`Data Entry Errors` , `Semi-Automated Data Validations`):** Automated extraction reduces typos; Pydantic validation enforces data integrity.

- **Consistency (`Inconsistent Information`):** Validation rules and Neo4j's relationship modeling help flag discrepancies.

- **Speed:** The workflow aims for minute-level processing instead of days.

- **Objectivity (`Subjective Decision-Making`):** The ML model provides standardized, explainable scoring.

- **Multimodal & Agentic:** Meets core technical requirements using LangGraph and Ollama.

- **Local Processing:** Ensures data privacy by keeping LLM inference local.

- **Targeted Support:** RAG enables relevant economic enablement recommendations.
- **Transparency:** Langfuse Cloud provides visibility into the automated process.

---

5. Challenges Faced During Development

- **Dependency Management:** Encountered significant conflicts between different versions of Langchain ecosystem packages (`langchain` , `langchain-core` , `langchain-community` , `langgraph` , `langchain-ollama`), requiring careful pinning of compatible versions (`requirements.txt`) to achieve a stable environment.
- **Initial Parsing Strategy:** Attempts to use complex OCR/Layout libraries (`paddleocr` , etc.) resulted in prohibitive installation times and platform compatibility issues. Pivoted to a V1 strategy leveraging Ollama's VLM capabilities directly via its Python client, combined with `pdfplumber` , which proved much faster to set up and satisfied the multimodal requirement effectively.
- **Langfuse Self-Hosting:** Faced persistent configuration errors (ClickHouse URLs, internal Zod validation errors for SALT/ENCRYPTION keys) with the `ghcr.io/langfuse/langfuse:latest` Docker image. Troubleshooting proved time-consuming under the deadline, leading to the decision to switch to Langfuse Cloud for reliable observability in the prototype.

---

6. Future Improvements & Integration

- **Enhance ML Model:** Train on a larger, realistic dataset; implement SHAP for decision explainability; add fairness/drift monitoring.
- **Refine RAG:** Populate Qdrant with real job/training embeddings; use applicant's resume vector for querying; employ an LLM to synthesize RAG results into actionable recommendations.

- **Implement HITL:** Route borderline eligibility scores or validation flags to a manual review queue with a dedicated interface.
- **Chat Agent & Clarification Loop:** Implement a true conversational agent in Streamlit; add a LangGraph loop where validation errors trigger the agent to ask the user for clarification.
- **Document Fraud Detection:** Add basic checks (metadata, simple image analysis).
- **API Integration:** Define specific APIs for receiving applications from external systems and sending status updates/decisions downstream.
- **Async Frontend Updates:** Use WebSockets/SSE instead of polling in Streamlit for real-time status updates.
- **Security & Compliance:** Implement robust AuthN/AuthZ (e.g., OIDC), PII encryption-at-rest (`pgcrypto`), and detailed audit logging.
- **Configuration Management:** Externalize thresholds, rules, and prompts into configuration files or a dedicated service.